Web And Cloud Based Security

- **50%** Cloud Security Assignment
- **20%** Rob The Bank (Penetration Testing)
- **30%** RobPress (Defend against Attackers)

Cloud Introduction

Advanced Persistent Threats (APT)
- Command & Control
- Process injection
- Cloaking
- Exfiltration
- Reignition

Pervasive Monitoring
- PM is widespread (often covert) surveillance through intrusive gathering of protocol artefacts, including application content, or protocol meta-data such as headers.

Security by Design
- Implementation of multiple security measures to protect the most valuable resources on your network
    - Network, Code, People

Cloud Services

Developing with security in mind from the start will reduce overall development time.

Security Terms
- **Monitoring practices**: Storage analytics and security logs
- I**dentify/access control**: RBAC (Role-based), MFA and password policies
- **Network security**: VPNs and network security groups
- **Store security**: Data encryption, client-side encryption and key vaults

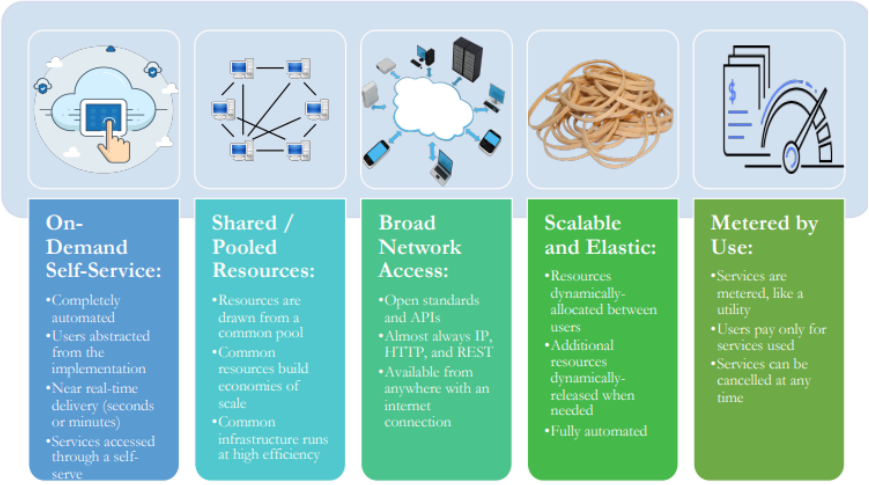
What is Cloud Computing?
- “A <u>model</u> for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction”

On-premises computing vs cloud computing:

On-premises	Cloud
Manually Provisioned	Self-provisioned
Dedicated Hardware	Shared Hardware
Fixed Capacity	Elastic Capacity
Pay for Capacity	Pay for Use
Capital & Operational Expenses	Operational Expenses
Managed via Sysadmins	Manages vis APIs

Five essential **characteristics** of cloud computing:

- On-demand self-service
- Shared/Pooled Resources
- Broad Network Access
- Scalable and Elastic
- Metered by Use



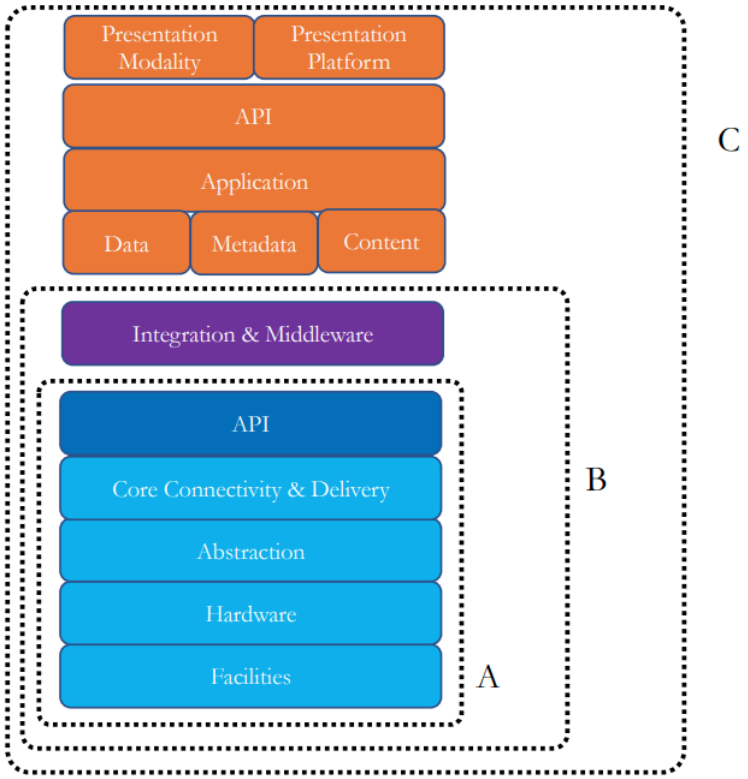
Cloud Benefits

- Cost efficiencies
- Time efficiencies (Elastic)
- Power efficiencies
- Improved process control
- Improved security
- “Unlimited” capacity

Deployment Models

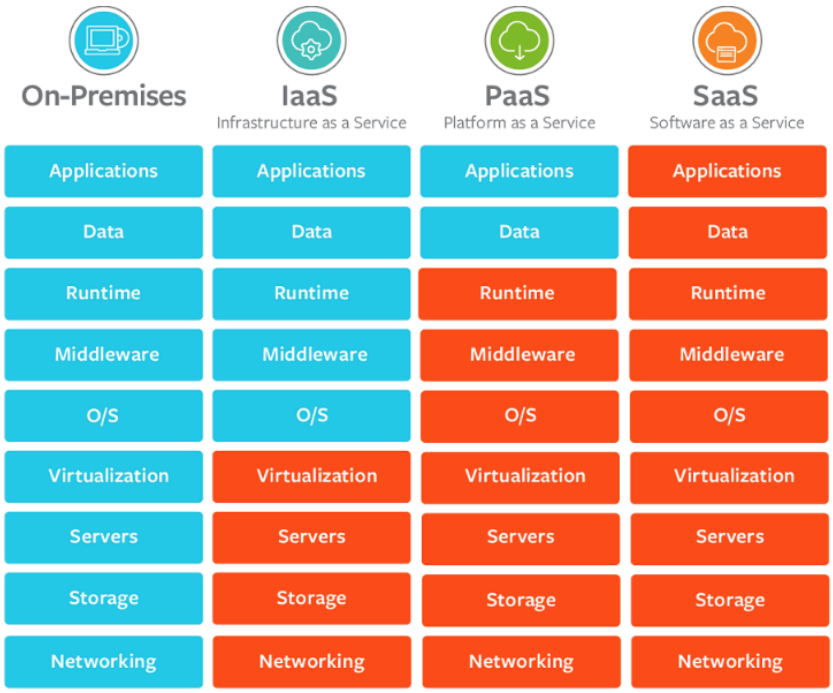
- **Public**: Infrastructure available to the general public, own by org selling cloud services
- **Private**: Infrastructure for single org, managed by org or 3rd party (on/off premises)
- **Community**: Infrastructure shared by many org, that have shared concern.
- **Hybrid**: Combination

CSA Cloud Reference Model



C: SaaS, B: PaaS, A: IaaS

Main Service Delivery Models



- **IaaS**: Consumers can provision resources and can deploy software, including OS and applications. VMs, Virtual Networks.
- **PaaS**: Consumers can create applications using tools supported by the provider. Auto Elastic, Continuous Integration
- **SaaS**: Consumer uses provider's applications on their infrastructure

Landscape and Policy

What is a Policy?
- Network - **Policy**: If you can measure it
- Code - **Practice**: If you are building
- People - **Guidance**: If you can't measure it

Policy → It is a statement that reflects the rules that everyone should abide by. All good policies are organised as follows:
- Purpose
- Policy Compliance
- Date Tested
- Date Updated
- Contact

Framework → Frameworks are like a toolbox, a set of tools.(Ready-made components or solutions)

Models → Step by step instructions (most likely abstract)

Measuring Security

When measuring performance of IT system you need:
- Goals
- Questions
- Metrics (Scales)

SANS Endpoint Security Maturity Model Curve:
- Level 1: Random, or Disorganised
- Level 2: Reactive or Tactical
- Level 3: Preventative
- Level 4: Organised, or DIrected
- Level 5: Proactive, Comprehensive, Continuous and Measurable

The Cybersecurity Capability Maturity Model (C2M2) maturity indicator levels:
- MIL0: Not accomplishing objectives or accomplishing with manual process

- MIL1: Accomplishing objectives, but with some automation, but minimal or ad hoc process
- MIL2: Established and followed standard operating procedures, more automation
- MIL3: Mature implementation with high degree of automation and highly optimised

Reference Diagrams (Of your network)

1. Define Hosts
2. Define Network
3. Define Stakeholders

Why?

- Show forbidden relationships
- Add monitoring nodes
- Identify host assets that needs to stretch
- Identify persistent assets

Host Assets → Typically equipment that have an OS and typically no firmware

- Servers and clients

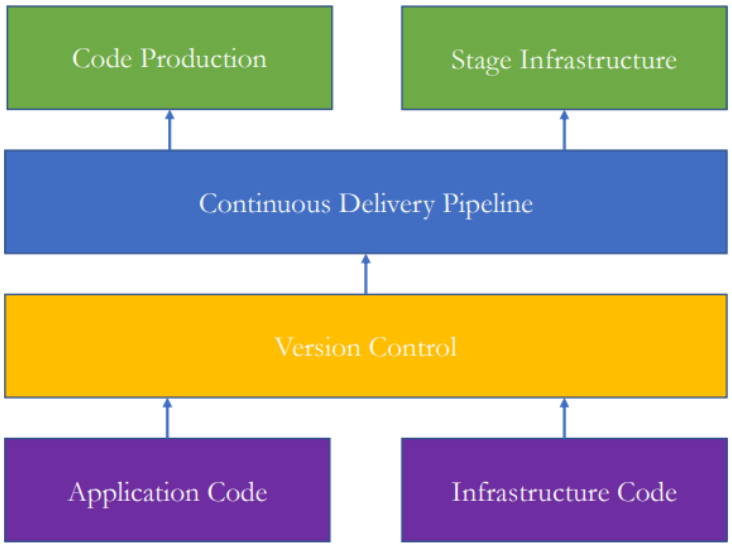
Network Assets → Assets that enable communication (e.g. wireless network, or virtual network)

Secure Cloud

Infrastructure as Code

DevOps → Network operations + Code development = People Culture

- **Development** → Developers; Front End; Quality Assurance
- **Operations** → System Admins; Network Admins; Database Admins



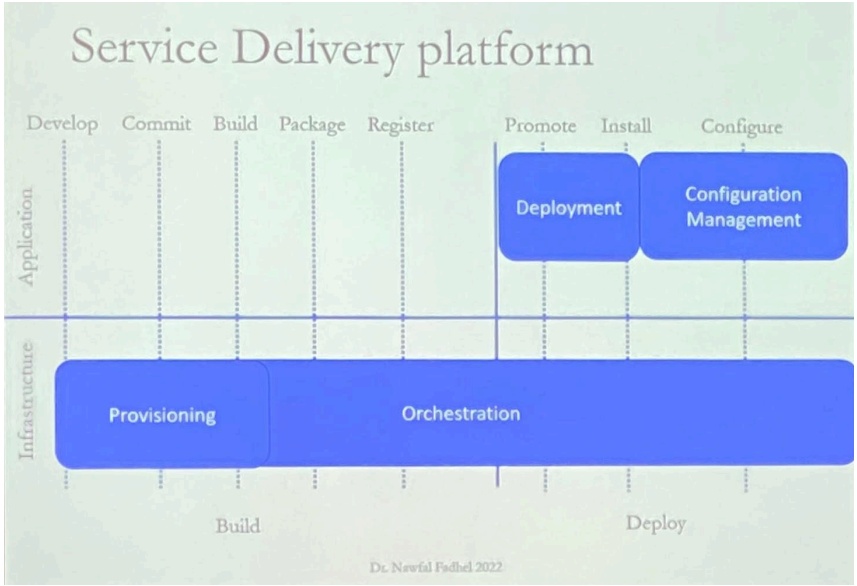
Infrastructure creation

Automatically provisioned	**Manually Provisioned**
Infrastructure is designed using code	Infrastructure is designed using graphs
Infrastructure is written using high-level programming language	Infrastructure is written using operations manual
Infrastructure can be tested and reviewed like code	Infrastructure can be tested and reviewed after deployment
Infrastructure can be relocated	Infrastructure can not be relocated
Reduction adding new bugs	Introduces new bugs
Development speed increases	Slows development speed
One employee can manage a big infrastructure	A team is needed to manage sometimes small infrastructure
You can repeat is exactly	You can not repeat it exactly
You can change it easily	You can not change is easily

Terms

- **Artefact** → Are what gets, versioned, tested and deployed
- **Procedures** → (Imperative), the commands to produce designed state are defined and executed (E.g. Bash Script)
- **Functions** → (Declarative) The desired state is defined, the tool conforms the system to the model of the desired state (E.g. SQL queries and make files)

- **Provisioning** → The process of making a server reading for operation, including hardware, OS, system services and network connectiving (Infrastructure)
- **Deployment** → The process of automatically deploying and upgrading applications on a server (Application)
- **Orchestration** → The act of performing coordinated operations across multiple systems. Initial provision is done once, being able to control and upgrade a running system is critical (Infrastructure)
- **Configuration Management (CM)** → The act of maintaining and upgrading application and application dependencies (Application)



Testing for Infrastructure as Code

YAML → Language that is often used for writing configuration files

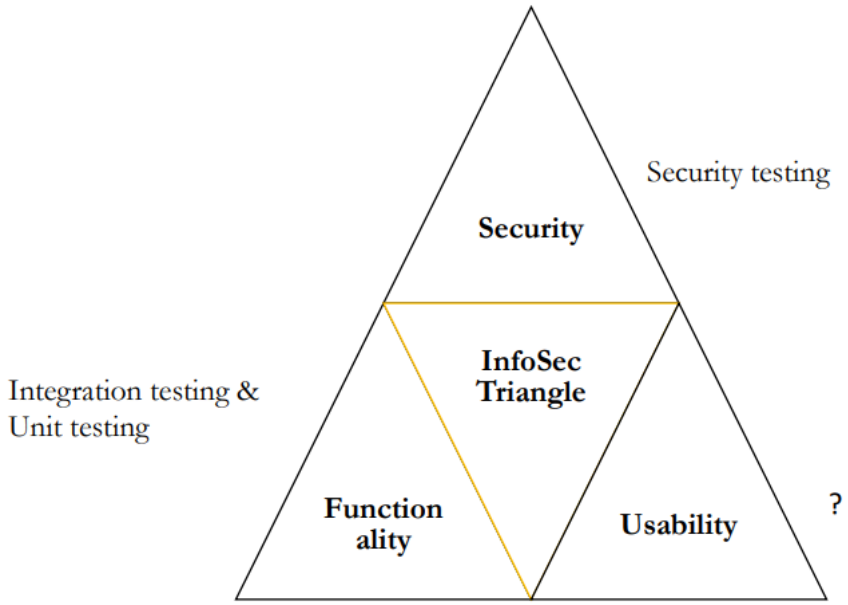
- YAML Scripts can define how to create a VM
- Can create VMs on more than one server
    - Allows company to manage all of their machines from a single interface
- Docker containers work on the same idea

Testing

    - Infrastructure Testing
- **Unit Testing**
    - Code-level testing and takes the smallest testable code blocks and verifies their execution. We lump formatters and linters in this category.
- **Integration Testing**

    - Heart of infrastructure. Focuses on testing several components together with a fully assembled and running system. (The build and runtime has to be validated)
    - Testing that the infrastructure integrates together
- **Security Testing**
    - Looking for both positive and negative security attributes. Can check for expected hardening of the system or test for possible weak spots

InfoSec Triangle



- Companies could remove features to have a smaller code base and therefore less areas of attack (improving security). But usability and functionality would decrease.
- There is a compromise between security, useability and functionality

Security Engineering
- Increasing Security can lower usability and vice versa
- b

Vulnerability and Threats

- **Vulnerability** → The weakness that enables the threat to be successful
    - E.g. log4j, no encryption, missing authorization, etc…
- **Threat** → A circumstance or event that could damage the confidentiality, integrity, or availability of information or information systems. (and events that could damage other factors such as authorisation etc…)
    - E.g. DDOS attack, SQL Injection, etc…

Virtual Machines

- Problems that can be solved using virtualization:
    - Efficiency
    - Scalability
    - Easy setup and configuration/Easy to duplicate configuration
    - Isolates deployments

- **Virtualisation** Software vs. **Virtualized** software
    - Virtualisation Software → Virtualises hardware or software: Virtual Box, VMWare
    - A-Virtualized Software → *Native* cloud app: Google Sheets
    - B-Virtualized Software → Virtualization software that is *not meant to be virtualized*, e.g. trying to run legacy software

- Concept or Native or “bare metal” hypervisor - No OS between virtualisation software and the hardware
- Concept of Host based

- You can use direct physical storage and/or virtual disks on virtualized resources

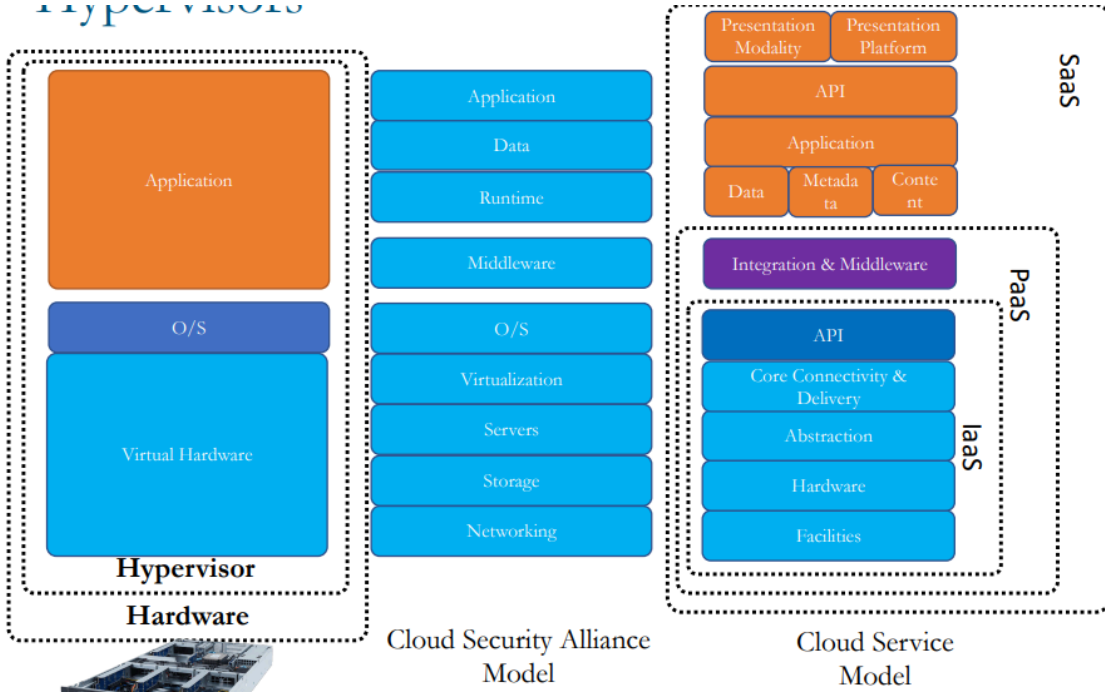
Virtualization Types

1. Hardware
2. Software
3. OS
4. Server
5. Storage

- Virtualisation is mainly used for IaaS and PaaS

Categories of Virtualisation

- Workstation Hypervisors → Creates, runs and manages VMs
    - VMWare workstation
    - VirtualBox
- Containers → OS-level virtualisation, emulates an OS instead of hardware
    - Docker
    - LXC
- Enterprise Hypervisors → Creates, runs and manages VMs
    - VMWare EXCI/VCenter
    - Microsoft Hyper-V
- Container Management Solutions → Automates the create, deployment and scaling of containers
    - Containerd
    - Kubernetes
    - Vmware integrated containers (VIC)

- Hypervisors
- 

- Have to install for IaaS

Hypervisor Vulnerability

- Virtual Machine Escape → Process of a program breaking out of the virtual machine on which is is running and interacting with the host operating system
- HyperJacking → Involves installing a malicious, fake hypervisor that can manage the entire server system
- Meltdown → Breaks the most fundamental isolation between user applications and the operating system. Allows a program to access the memory, and thus also secrets of other programs and the operating system
- Spectre → Breaks the isolation between different applications. Allows an attack to trick error-free programs, which follow best practices, into leaking their secrets. In fast, the safety checks of said best practices actually increase the attack surface and may make applications more susceptible to Spectre

Dockers

Setting up a Web Application:

- Backend: node.js and express API
- DB: MongoDB
- Frontend: Apache

Setting up across different systems, projects or developers

- Multiple developers working on a project need the same resources in their local environment for testing purposes
    - How can you guarantee to run the same environment resources and versions for each developer?

Docker vs Hypervisor

- A Hypervisor will virtualise the hardware, whereas a docker uses OS-virtualization

Setting up a docker container:

1. # Set the base image to Ubuntu
   FROM ubuntu
2. # Update the repository sources list and install gnupg2
   RUN apt-get update && apt-get install -y gnupg2
3. # Install MongoDB package (.deb)
   RUN apt-get install -y mongodb
4. # Create the default data directory
   RUN mkdir -p /data/db
5. # Expose the default port
   EXPOSE 27017
6. # Default port to execute the entrypoint (MongoDB)
   CMD ["--port 27017"]
7. # Set default container command
   ENTRYPOINT usr/bin/mongodb

-

Container Contains

- Code
- Config
- Processes
- Networking
- Dependencies
- NOT an OS

Dockers

- Start, stop and restart containers if they fail
- Help dockers discover other services
- Allocation resources

Docker Vulnerabilities

- Software Vulnerability
    - *Container Escape*: the attacker gains full control of the host and can try to attack resources in the internal network segment. This can allow an attacker to deploy a malicious container inside the production environment

Service Level Agreement

- SLA is a part of a contract which defines exactly what services a service provider will provide and the required level or standard for those services
- SLA is generally part of an outsourcing or managed services agreement. Can be used in facilities management agreements, and other agreements that are for the provision of services

SLA → Sets out overall objectives for services to be provided. E.g:

- Improve performance
- Save costs
- Provide access to skills
- Provide access to technologies

    - These, if done internally, would normally cost a lot (E.g. hiring a person to maintain the software as well as the infrastructure behind it, so offload this responsibility to cloud provider)

- **Description of the Services**
    - SLA should include a detailed description of the services
    - Each individual service should be defined

- **Performance Standards**
    - Customer will want highest performance, in reality this is impossible, unnecessary or very expensive
    - Service provider may argue that service levels should be set low in order to guarantee that the service can be set at a competitive price

- **Compensation/Service Credits**
    - Incentive for service provider to have less failure, may have financial consequence for service provider
    - Most often achieved through the inclusion of a service credit regime. In which, where the service provider fails to achieve the performance standards, the provider will pay or credit the consumer an agreed amount

- **Critical Failure**

    - A right for customer to **terminate** agreement if service delivery becomes unacceptably bad
    - A right to sue for damages
    - A right for service credit

- Service is a means of delivering value to customers by facilitating outcomes customers want to achieve without the ownership of specific costs and risks

- Main elements of a good SLA:
    1. Overall objectives
    2. Description of the Services
    3. Performance Standards
    4. Compensation/Service Credits
    5. Critical Failure

- Service Level Management
    - Understand, Negotiate and Agree
    - Deploy SLA
    - Monitor, Report and Improve

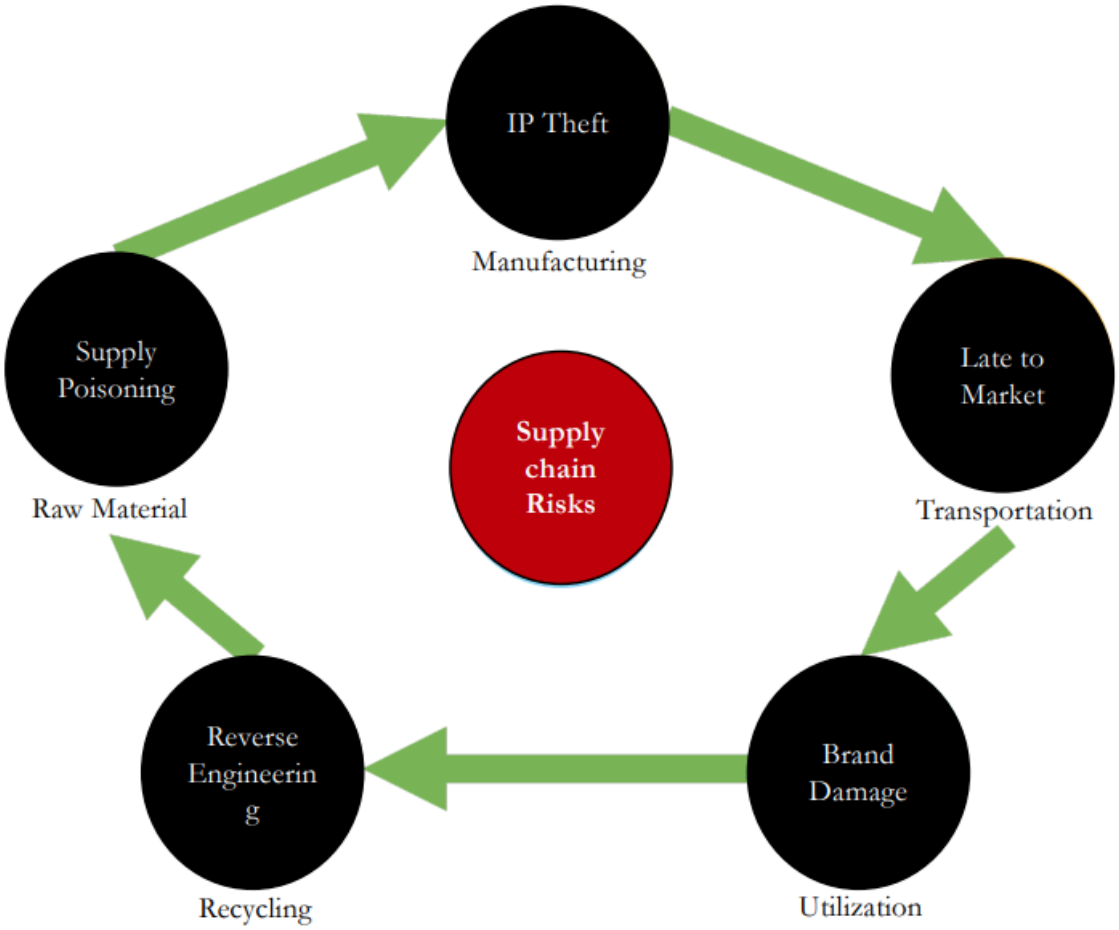
- Service Provider, if they draft the SLA they will write it in favour of them

Cloud Vulnerabilities

- Attack vectors - Goal of the attack (E.g. breach user privacy)
- Threats - How you would do that (E.g SQL Injection)
- Vulnerability - Why that is possible (E.g. Input not sanitised)

Assume every connection is vulnerable
Choose highest assets
Choose threats

Attack Vectors



Threats

1. **Accountability and data ownership**
    a. Consider data toxicity
    b. Who owns the data
    c. Where to store it (isolation)
    d. How to destroy it
    e. How to encrypt it
2. **User Identity Federation**
    a. Managing identities across multiple providers
    b. Less control over user life cycle
    c. User experience
    d. Identity providers:
        i. SAML, Open ID (Oauth), Access Control Design
3. **Regulatory Compliance**
    a. Data that is perceived secure in one region is not assumed secure by another

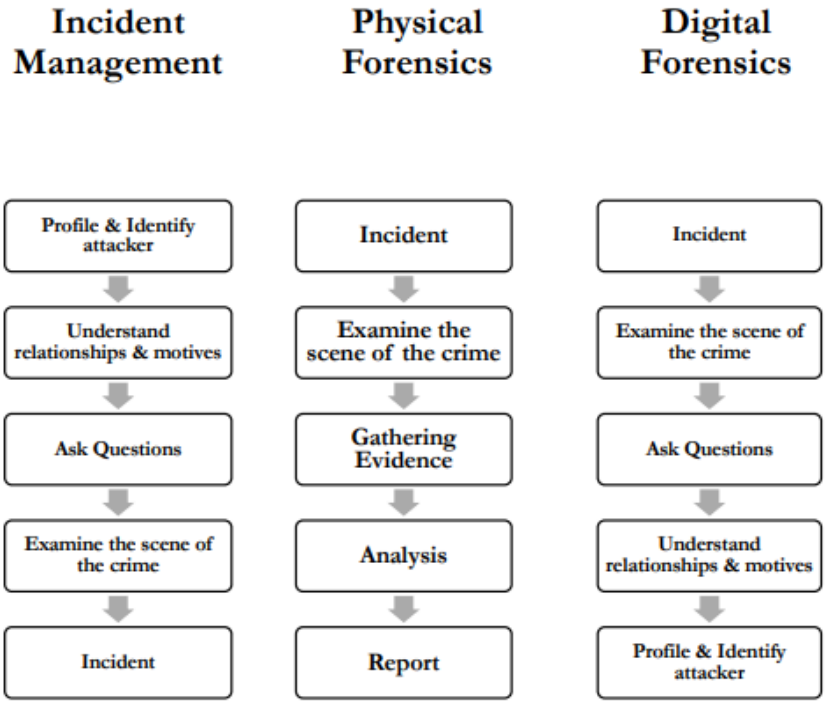
   b. Lack of transparency in the underlying implementations makes it difficult for data owners to demonstrate compliance to owners
   c. Lack of consistency global standard for handling data
      i. <u>EU has very strict privacy laws and hence data store in the US may not comply with EU law</u>
   d. Use a cloud vendor who understands and applies solutions for the various data protection laws
   e. They should also know how to handle cross-jurisdiction data protection requirements
4. **Business Continuity and resiliency**
   a. Need to make sure that your Service Level Agreements (SLAs) cover data resilience, protection, privacy, and that the vendor has a robust disaster recovery process in place
5. **User privacy and secondary usage of data**
   a. User Priority
      i. Keep personal data safe
   b. Providers Priority
      i. Push out the liability to user via privacy and acceptable use policy
      ii. Build addition services on user behaviour
      iii. Do the minimum to achieve compliance
      iv. Keep their social application more open
   c. Anonymize personal information
   d. Encrypt storage
   e. Terms of service with providers (Compliance and Geographical restrictions)
6. **Service and data integration**
   a. Data in Transit → SSL/TLS
      i. Authentication
      ii. Authorization
      iii. Accounting
   b. Data at Rest → Encryption keys
      i. Confidentiality
      ii. Integrity
      iii. Availability
7. **Multi tenancy and physical security**
   a. Concerns
      i. Inadequate logical separations
      ii. Co-mingled tenant data
      iii. Malicious or ignorant tenants
      iv. Shared service, single point of failure
      v. Uncoordinated change controls and misconfigurations
      vi. Performance Risks
   b. Possible attacks
      i. Cross-tenant attacks
      ii. Side channel attacks

iii. Scanning other tenants
iv. DOS

c. Solution
   i. Build architecture for multi-tenancy
   ii. Data encryption (per tenant)
   iii. Controlled and coordinated change management
   iv. Ability to audit the provider administrative access
   v. Third party assessment

8. Incident analysis and forensic support

a. If data breach occurs, need to understand how to identify and manage critical vulnerabilities so you can respond to the incident quickly and effectively. Cloud computing can make the forensic analysis of security incidents more difficult, this is because audits and events may be logged to data centres across multiple jurisdictions.
b. Check cloud vendor policy on handling, evaluating and correlating event logs across jurisdictions
c. Do cloud vendor have technologies in place, such as virtual machine imaging, to help in the forensic analysis of security incidents
d. Investigator teams
   i.



9. Infrastructure support

a. SANS Endpoint Security Maturity Model Curve
   i. Level 1: Random or Disorganised
   ii. Level 2: Reactive or Tactical
   iii. Level 3: Preventive
   iv. Level 4: Organised or Directed

        v. Level 5: Proactive, Comprehensive, Continuous and Measurable
    b. Infrastructure Security
        i. MIL0: Not accomplishing objectives or accomplishing with manual process
        ii. MIL1: Accomplishing objectives, but with some automation, but minimal or ad hoc process
        iii. MIL2: Established and followed standard operating procedures, more automation
        iv. MIL3: Mature implementation with high degree of automation and highly optimised

10. Non-production environment exposure

    a. Don't use the cloud to develop concepts or tests
    b. Always use MFA
    c. Treat any pre-production environment or data as top secret

- Encryption at rest → Encrypt data when stored

Legal Aspects

Web Vulnerabilities

Main Vulnerabilities we will we exploring

- SQL Injection
- Information Exposure
- XSS
- Insecure Upload
- Cross-Site Request Forgery
- Authorisation Bypass
- Internal Information
- Parameter Manipulation
- Application Logic
- Out of Date Software
- File Inclusion
- Insecure Cookies and Sessions
- Open Redirects

- Brute-force
- Insufficient Logging

Three Big Vulnerabilities

- Cross-site scripting
- SQL Injection
- HTML and form manipulation

- People usually won't use the website in the way you want them to
    - May not follow the steps, press the worst buttons, put in wrong information and ignore processes etc…

- Early web was rife with vulnerabilities
- Now there are still vulnerabilities

- Not just web applications
    - APIs, Mobile Apps, everything…

How do applications work?

HTTP

- GET request for a webpage
- Server-side applications → Core of application lives in server
- Client-side applications → Core of application lives in browser/client

- HTTP verbs
    - GET → read
    - POST → create
- Usually in RESTful services:
    - PUT → update/replace
    - PATCH → update/modify
    - DELETE

- HTTP status codes
    - Informational **1xx**
    - Success **2xx**
    - Redirect **3xx**
    - Client error **4xx**
    - Server error **5xx**

HTML

- What are the most useful tags to us? → Ones involving user interaction

        - <a>
        - <form>
        - <input>, <select>, <textarea>, <button>
        - <script>

Other Web Languages
- Client-side
    - CSS
    - JavaScript
    - Web Assembly (wasm)
- Server-side
    - PHP
    - Python
    - Perl
    - etc…
- Communication
    - JSON
    - XML
    - Raw
- On-Going Communications
    - Simple AJAX / XHR (XMLHttpRequest)
    - AJAX Polling
    - WebSockets (Possibly best option)
    - WebRTC

How do we pass in user input?
- GET query parameters (In URL, after ?)
- POST form data (Not in URL)

User input
- URL parameters (/value/value)
- GET parameters (?key=value)
- POST parameters (forms: key=value)
- Logic and series of interactions
- Cookie values
- Headers

- User input is what gets processed by the application - through this, we control the application

Cross-Site Scripting (XSS)

- Major Types
    - **Reflected**

        - Where a user input to a page can be populated with XSS and gets output as a result when that page displays
    - **Stored**
        - Where a user input is written or stored and later displayed back as the result of a query

- E.g. user input relays input back to user, allowing scripts to be run
- We can run any javascript we want!
    - Get the current content of authentication cookie
    - Control or manipulate the page in any way
    - Control of manipulate the user's browser
    - Make requests as if we were the user's browser
    - Make outgoing requests

SQL Injection

- Manipulating the query sent between the server and the database (using SQL)
- Can allow for SQL queries such as DROP Table, which is bad

HTML and Parameter Manipulation

- Manipulate inputs that client sends to the server
    - The URL itself
    - Forms and their parameters
    - JavaScript server requests and API calls
    - Communications

- How?
    - Using the browser
    - Client-side scripting
    - Download and modify
    - Using extensions
    - Scripts
    - Tools

Theory into Practice

Using HTML for bad

- We can take HTML from a good side, modify it, and make it bad (AKA: steal information)
- Common part of phishing attacks, what can they get?
    - Money

    - Information
    - Access
    - Resources
    - Fun
    - Publicity

Phishing

How does phishing work?

- **Impersonation**
    - Sending false emails/messages
    - Setting up false websites to steal information
    - Using phishing kits / perverse proxy sites

    - Set up a website that looks like the legitimate website
    - Add forms to collect login information or other personal information, in the same manner as the legitimate site
    - Try to disguise the website to make it look like the legitimate website
    - Trick users into using website, typically by sending email with link …

    - Phishing Collector
        - Retrieves the username, password or any confidential information entered into the site
        - Stores it for the person running the phish to collect later
        - Redirects the user back to a legitimate page (So they don't realise they've been phished)
- **Server Hijacking**
    - Manipulating the legitimate website to steal information

    - Exploiting a legitimate website to turn it into a phish
    - Typically involves changing the forms on the website to point to the phishing collector instead of where they should
    - Three major forms of server hijacking
        - Cross-site scripting
            - Allow us to "reprogram" the website by injecting additional code
        - Open redirects
            - Being able to "bounce" through a legitimate website to the phishing site, so it looks legitimate
        - Direct compromise
            - Gaining access to the server and being able to rewrite files and manipulate them directly
- **Client Hijacking**
    - Manipulating your computer to steal information

Cross-Site Scripting and Phishing

- We can change HTML code on the website even after it has loaded

- Typically:
    - Change where the form on the page goes to
        - Point it to our collector
    - Add a new form onto the page that we control
        - And hide anything else

- Insert scripts which change form action, will send data to our phishing website

Information Disclosure

- When sensitive information is accidentally exposed by an application
    - It may be user data - e.g. another user being able to get more information about another user
    - Application data, e.g. leaked resources, version information, configuration details
    - Deployment information - e.g. URLs or structure
- As black block tester, scrutinise all outputs from the application for any information you can use
    - Can be tedious

- What can you find with the information the server sends you.

- Look for hidden parts of webpages using developer tools

Authorisation Bypass

- When authorisation is not implemented properly
- Sometimes is possible to bypass checks
    - Authorisation by omission
        - Some applications hide administrative controls from non-admins, but do not check server-side (only need the link)
    - Missing authorisation throughout the process
        - Some applications check authorisation at the start of the process but not all the way through - bypass the initial check
    - “Safe” areas
        - Some applications have safe areas where it is presumed everything inside has already passed authorisation checks

- Can guess URL, such as /admin

Insecure Cookies

- Cookies are used by applications to store client-side information
    - Typically session or user key
    - Required so that the applications continues to know who you are after you have logged in
    - When you log in, you typically get set a cookie containing some form of key which identifies you
- Cookies are sent in the headers with **every** request you make
    - Password is not what identifies you - the cookie is
- Cookies can be modified by the client and can be insecure
    - If a cookie is stolen they can be used by another user without password

- If authentication cookies aren't set randomly then others can be guess
    - E.g. sequential cookies are bad

Robbing the Shop

Process of Penetration Testing

- Scoping
    - Agree on what is being tested, when and how
    - Define boundaries
- Gather information
    - Infrastructure
    - Application (logic)
    - Mapping
    - URL structure
    - Inputs and outputs
    - Authentication / Authorisation
    - Validation
    - Error handling
    - Sessions and Cookies
- Identify vulnerabilities
- Exploit: Elimite false positives
- Post-exploit: What can you do now?
- Report

Things to think about:

1. By systematic
2. Be logical

3. Map out what you're doing
4. Use tools to help you
5. Be the anti-user
6. Intercept and Manipulate
7. Cheat - Rob The Shop has firewall in front of it, so don't spam
8. Don't give up

Burp Suite
- An intercepting proxy
- Everything passes through it
    - It collects information
    - It allows you to track what you have done
    - It allows you to review previous requests and responses
    - It lets you manipulate everything
- Useful for bypassing client-side validation
- Useful for being repeatable

No Spam
- No more than 100-200 requests at a time
- Safe throttle pool is:
    - Max concurrent - 2
    - Delay between requests - 300ms
    - Random variations

- Use Kali

Burp Suite
- Spider
    - Scans page and visits all URLs on that page
- Intruder
    - Automates putting in inputs
    - Configure payloads
    - Fuzzing
    - Grep error or logged in
- Sniper

- SQLMap
- Dirb

Be careful
- Avoid triggering intrusion detection or firewalls
- Don't use dirb or content discovery on Rob the Bank with giant lists

Keep track of where we can enter data

- URL
- GET parameters
- POST parameters
- Session and Cookies etc…

XSS

- Input that becomes a unescaped output

Map out what you're doing

Possible way to write it down

URL	GET Parameters	POST Parameters	Authorisation	Features	Vulnerabilities & Comments
Parameter / S / X / M	Parameter / S / X / M	Parameter / S / X / M	Role / E / A	Feature / ? / V	
				DB Read	
				DB Write	
				Logic	
				CSRF	
				Upload	
				Email	
				SetCookie	

URL	GET Parameters	POST Parameters	Authorisation	Features	Vulnerabilities & Comments
/users/view/<id>	Parameter / S / X / M	Parameter / S / X / M	Role / E / A	Feature / ? / V	Parameter Manipulation + Internal Information: Error produced when type parameter is invalid with stack trace
Parameter / S / X / M		name	Admin / X / X	DB Read	
<id>		type / X	User / X	DB Write / X	Authorisation Bypass: User can edit profile.
			Guest	Logic	
				CSRF / X / X	Vulnerable to CSRF and file upload
				Upload / X / X	
				Email	
				SetCookie	

-

More Vulnerabilities

- Not assessed
    - Mapping sheet

Mapping

- What pages are there?
- What forms are there?

- What does it take user input?
- What features are present?

- Map Rob the Bank
    - Map before finding vulnerabilities
    - Complete this by *monday*
    - Only need to map every URL and the associated URL, GET and POST parameters
- Provided with basic mapping template

Internal Information

- Information which the app gives you, which it shouldn't

- Error reporting, debug information, unexpected behaviours
- You are trying to create those faults, those situation, those errors

- The attacker needs information, do not give them information
- Do NOT give them any information.

Finding leftover files

- Look for common files with common names
    - Testing, phpinfo, info, debug etc…
- Look for variations
    - Index.bak, index.php, index.zip etc…
- Hidden files
    - .git etc…

Parameter Manipulation

- Parameter manipulation and insufficient validation
- Client-side validation is insufficient
    - Client-side scripts, form controls
    - A combo box to limit choices
- Never trust the client!
    - Client can send any response it wants to the server, regardless of client-side validation
- What to look for?
    - Work through all the form fields

    - Try manipulating all form elements and parameters and see if you can pass data they shouldn't allow
- What to think about
    - What validation is being done?
    - Is it possible to get data in that shouldn't be there?

- What to try?
    - Different data types
    - Out of range
    - No input
    - New parameters
    - Overflow
    - Injection
    - Execute commands

Application logic

- Look for
    - Where user input is *processed* not just queried/stored
    - Multiple-step processes (Bypass user login etc…)
    - Any parts of the application which the user can manipulate
    - Is there any application logic which the users can affect through user input

- Areas of the application which perform logic

Out of Data Software

- Responsible for so many compromises
- Multiple layers
    - Web applications
    - Libraries
    - Operating System

- Look for
    - Out of data software
    - Out of date component or underlying software
    - Signs of what software the site is using
    - Signs of the version they are using
    - Any vulnerabilities in that version

National Vulnerability Database

- Database of known vulnerabilities in software
- CVE numbers - unique identifiers for vulnerabilities

File Inclusion

- Look for
    - Places in which files from the file system are included including user input
- Where a file is dynamically included by the URL
- Good file to aim for:
    - ../../etc/passwd
- E.g. *stylesheet.php?style=blue.css*

Open Redirects

- Look for
    - Any redirects in the application
    - Which pass the redirect through the URL
- Can these URLs be redirected to other pages
- 3xx status codes
- Look at request

- URL Encode

Insecure Cookies and Sessions

- Sessions and Cookies
- Incremental cookies and sessions

- Cookies
    - Manipulate them
    - Investigate
    - Are they are hash or encoded?

- Don’t worry about CAKEPHP cookie

Insecure Upload

- Users can upload arbitrary content to your website
    - Could set up their own phishing site on your domain
    - Execute code on your server
- Look for upload forms

- Filename → Does it change the filename you got it
- MIME types
- File extensions → Can you confuse it
- Content → A valid image?
- Still need it to be effective
    - A .jpg not going to be executable

Server-SIde Request Forgery (SSRF)

- Getting the server to make a request
    - Via API or HTTP call

Secure Coding